The Role of DevOps and CI/CD in Automating Research Projects: the Case of a Retrieval-Augmented Generation System for Intelligent Enterprise Search

Medet Kabdulinov

Assignment 5. DevOps and CI/CD: Automating Processes for Research Projects

Introduction

My master's thesis is about a Retrieval-Augmented Generation (RAG) system for intelligent enterprise search. Such a system answers an employee's question in two steps. It first retrieves relevant passages from corporate documents, and then a large language model (LLM) writes an answer grounded in those passages [1]. The system is a chain of components: chunking, a retrieval index, a prompt, an external model and an evaluation set. Each of them changes often during research, and any change can silently make the answers worse. This report explains DevOps and CI/CD and shows how they apply to the thesis prototype.

1. What is DevOps: goals, principles and benefits

DevOps is a culture and a set of engineering practices that remove the wall between those who write software (development) and those who run it (operations). The term appeared around 2009 with the first DevOpsDays conference organised by Patrick Debois, as a reaction to slow, manual and risky releases. Its central goal is to deliver changes to users **quickly, safely and repeatedly**.

The principles are often summarised by the acronym **CALMS**:

- **Culture** — shared responsibility for the product from the first commit to production;
- **Automation** — everything that is done more than twice (build, test, deployment, infrastructure setup) is done by a machine;
- **Lean** — small batches of change, short feedback loops, limiting work in progress;
- **Measurement** — decisions are based on data: build times, failure rates, quality metrics;
- **Sharing** — knowledge, code and configuration are open to the whole team.

Research by the DORA group, summarised in the book *Accelerate* [2], showed that four metrics separate high-performing teams from the rest: deployment frequency, lead time for changes,

change failure rate and time to restore service. Teams that automate their delivery deploy more often *and* fail less often, which contradicts the intuition that speed costs stability.

The main benefits of DevOps are faster feedback, fewer human errors, reproducible environments, quick rollbacks, and a record of every change, test result and deployment.

2. Overview of CI/CD

Continuous Integration (CI) is the practice of merging every developer's changes into the main branch frequently, at least daily, where each change automatically triggers a build and a test suite. A broken build is visible within minutes and is fixed before other work continues. The key elements of CI are a version control system (Git), an automated build, a fast and reliable test suite, and a CI server that runs them on every commit.

Continuous Delivery (CD) extends CI: every change that passes the pipeline produces a release candidate — for example, a Docker image — which *can* be deployed to production at any moment by pressing a button [3]. **Continuous Deployment** goes one step further and deploys every successful change automatically.

A CI/CD pipeline is usually described as a sequence of stages:

Stage	Purpose
Source	a commit or pull request triggers the pipeline
Build	compile code, install dependencies, build an artefact or container image
Test	static analysis, unit and integration tests, quality gates
Package / publish	push the artefact to a registry with a version tag
Deploy	roll out to a test or staging environment, then to production
Verify	health checks, smoke tests, monitoring; roll back on failure

The pipeline itself is stored as code (a Jenkinsfile, .gitlab-ci.yml or GitHub Actions workflow) next to the application, so it is versioned, reviewed and reproducible like any other code.

3. The role of DevOps tools in research environments

Scientific software has specific problems that DevOps tools address directly:

Reproducibility. A result published in a paper should be obtainable again by another researcher, or by the same researcher a year later. In practice, results depend on library versions, operating system packages, random seeds and data snapshots. Containers fix the whole software environment in an image; pinned dependencies and data stored in version control fix the rest. Boettiger [4] argued that Docker directly addresses the most common causes of irreproducible computational research: dependency hell, imprecise documentation and code rot.

Reliability of results. Research code is often written quickly and tested manually. Automated unit tests catch regressions in algorithms; for machine-learning and information retrieval systems, the test suite can also include *quality gates* — a metric computed on a fixed evaluation set that must not drop below a threshold. This turns "the new chunking seems better" into a measured, recorded fact.

Collaboration. In a team or between a student and a supervisor, CI provides a shared, neutral judgement of whether the code works, independent of anyone's laptop. Code review and pull requests document why a change was made.

Hidden technical debt in ML systems. Sculley et al. [5] showed that in real ML systems only a small fraction of the code is the model itself; the rest is data collection, configuration, serving and monitoring infrastructure. A RAG system is a typical example: the "intelligence" is a few prompts and an index, while most of the risk lies in the surrounding pipeline. DevOps practices — and their ML-specific extension, MLOps/LLMOps — make that infrastructure explicit and testable.

For a RAG system specifically, DevOps adds the following concerns to a classic web service: the document index must be rebuilt reproducibly when the corpus changes; retrieval quality (Recall@k, MRR) and answer faithfulness must be evaluated on every change; external LLM APIs are rate-limited and can fail, so the system needs fallbacks; and API keys must be handled as secrets, never stored in code or logs.

4. Examples of using Docker, Jenkins and Kubernetes in scientific and engineering projects

Docker packages an application with all its dependencies into an image that runs identically on a laptop, a CI server and a cloud machine [6]. In science it is used to publish the exact environment of a paper (for example, through services like Binder, which build a container from a repository so that readers can run the authors' notebooks in a browser), and to distribute bioinformatics tools — the BioContainers project provides thousands of containerised tools for

genomics pipelines. On high-performance computing clusters, where Docker's root daemon is not allowed, the compatible Singularity/Apptainer runtime [7] executes the same images.

Jenkins is an open-source automation server (a 2011 fork of Hudson) with more than a thousand plugins and support for pipelines written as code. Its strength in research and engineering organisations is that it can run on-premise, close to private data and special hardware, without sending code to a third-party cloud. CERN software teams, for example, have long used Jenkins to run nightly builds and tests of large physics frameworks on many platforms; industrial engineering teams use it to run hardware-in-the-loop tests. Jenkins pipelines can call Docker for every step, so the build environment is itself reproducible.

Kubernetes orchestrates containers across a cluster: it schedules them, restarts failed ones, performs rolling updates and scales services up and down [8]. In research it hosts shared platforms such as JupyterHub for courses and labs, scientific workflow engines (Argo Workflows, Nextflow on Kubernetes) and ML platforms such as Kubeflow, which run training and evaluation pipelines as containers. Large research infrastructures, including CERN's private cloud, offer Kubernetes clusters as a service to their scientists. For small projects, lightweight distributions such as Minikube and k3s make the same deployment manifests usable on one machine.

Application in my project. The thesis prototype (repository github.com/mkbdlnv/enterprise-rag) uses all three tools:

- a **multi-stage Dockerfile** builds two images from one file: a test image with pytest and Ruff, and a minimal, non-root runtime image with a health check;
- a **Jenkins** server, itself described as code (plugins list, Configuration-as-Code YAML, Job DSL), polls the Git repository and runs the pipeline: lint → unit tests (published as JUnit reports) → retrieval evaluation (Recall@3 and MRR archived as a build artefact; the tests fail the build if Recall@3 < 0.9) → runtime image → deployment to a test server with an automatic rollback if the new container is not healthy → smoke test against the live service;
- a **Kubernetes** manifest describes the same service as a two-replica Deployment with readiness and liveness probes and rolling updates, ready for Minikube or k3s.

As a result, every experiment with the retriever or the prompt is a commit. Its effect on retrieval quality is measured and recorded, and if it passes, the change is live at https://rag.mkab.dev within about two minutes.

5. Conclusion: what problems DevOps solves in scientific teams

DevOps turns research software from a fragile, personal artefact into a reproducible, verifiable and continuously available system. Concretely, it solves the following problems:

1. **Irreproducible environments** — solved by containers, pinned dependencies and infrastructure as code.
2. **Silent regressions** — solved by automated tests and metric-based quality gates on every commit.
3. **Slow, manual and risky releases** — solved by pipelines that build, deploy, check health and roll back automatically.
4. **Lack of traceability** — every result is linked to a commit, a build number, a test report and an image tag.
5. **Knowledge locked in one person** — the pipeline, server configuration and deployment are documented as code that anyone in the team can read and rerun.

For a RAG system for enterprise search these benefits are especially important, because its quality depends on many interacting components and on external models that change over time. Automating evaluation and delivery is what makes it possible to experiment quickly while still being able to justify every result reported in the thesis.

References

1. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
2. Forsgren N., Humble J., Kim G. *Accelerate: The Science of Lean Software and DevOps.* IT Revolution Press, 2018.
3. Humble J., Farley D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.* Addison-Wesley, 2010.
4. Boettiger C. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1), 2015, pp. 71–79.
5. Sculley D. et al. Hidden Technical Debt in Machine Learning Systems. *NeurIPS*, 2015.
6. Merkel D. Docker: lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).
7. Kurtzer G. M., Sochat V., Bauer M. W. Singularity: Scientific containers for mobility of compute. *PLoS ONE*, 12(5), 2017.
8. Burns B., Grant B., Oppenheimer D., Brewer E., Wilkes J. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 2016.
9. Robertson S., Zaragoza H. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.